

[2025-2][인공지능]텀프로젝트#1보고서

2025.12.13

지능기전공학부 스마트기기공학전공 21011934 유혁재

[2025-2][AI][텀 프로젝트 #1] 실험 결과 보고서

주제: 센서를 이용한 환자 활동 상태 예측

1. 개요 및 문제 해결 조건 준수 여부

본 프로젝트는 IMU 센서의 CWT 변환 데이터를 이용하여 환자의 6가지 활동 상태를 분류하는 AI 모델을 구축하는 것을 목표로 합니다. 주어진 베이스라인 코드와 문제 해결 조건을 아래와 같이 충실히 준수하였습니다.

구분	요구 사항	준수 여부 및 내용
데이터 선정	가속도(acc) vs 각속도(gyro) 중 택 1	가속도(acc) 데이터 선정
전처리	1D → 2D 복원 및 Resize (224x224)	(N, 6400) → (N, 3, 50, 128) → (N, 3, 224, 224) 변환 완료
모델	ResNet18사용	torchvision.models.resnet18(pretrained=True) 사용
실험 조건	1. 동결(Freeze) / 2. 미세조정(Fine-tuning)	두 가지 조건에 대해 실험 수행 및 비교 완료
Hyperparam	Batch 32, Epoch 5, LR 1e-3, Adam	요구된 하이퍼파라미터 그대로 적용
Seed	Random Seed 고정	Seed 42 고정 (Numpy, PyTorch, Python 등)

2. 실험 설계

2.1 데이터 전처리 프로세스

제공된 csv 파일은 CWT 변환 후 1D로 펼쳐진 데이터입니다. 이를 ResNet18 모델의 입력에 맞추기 위해 다음과 같은 과정을 거쳤습니다.

1. **Reshape**: 1D 벡터(6400)를 원본 CWT 형태인 (50, 128) 2D 이미지로 복원했습니다.
2. **Stacking**: x, y, z 축 데이터를 합쳐 (3, 50, 128) 형태의 3채널 이미지를 생성했습니다.
3. **Normalization**: Min-Max 정규화를 적용하여 데이터 범위를 0~1 사이로 변환했습니다.
4. **Resizing**: Bilinear Interpolation을 사용하여 모델 입력 크기인 (224, 224)로 변환했습니다

2.2 모델 구조 및 학습 설정

- **Backbone**: ResNet18
- **Head (Classifier)**: 기존 FC Layer를 제거하고, 출력 노드가 6개인 nn.Linear(512, 6)로 교체했습니다.
- **Loss Function**: CrossEntropyLoss
- **Optimizer**: Adam (Learning Rate: 0.001)

3. 코드 수정 및 성능 개선 전략

초기 실험(베이스라인 코드)에서는 정확도가 약 0.61에 머무르는 문제가 있었습니다. 데이터 분석 결과, 입력 데이터의 스케일(Scale) 문제가 원인임을 파악하고 아래와 같이 코드를 수정했습니다.

3.1 문제 분석

- **현상**: 초기 모델의 리더보드 점수가 0.6189로 낮음.
- **원인**: acc_x_train_CWT.csv 등의 데이터를 분석한 결과, 값의 범위가 -0.005 ~ 0.007 등으로 매우 작고 음수가 포함되어 있었습니다. ResNet과 같은 사전 학습 모델은 일반적으로 0~1 사이(또는 정규화된) 분포를 가진 이미지를 학습하도록 설계되어 있어, 스케일이 맞지 않는 데이터에서는 특징 추출 능력이 급격히 떨어집니다.

3.2 수정 내용 (Min-Max Normalization 적용)

SensorDataset 클래스의 __getitem__ 메서드에 데이터를 0과 1 사이로 매핑하는 Min-Max 정규화 코드를 추가했습니다.

[수정 전 코드]

```
img_tensor = torch.tensor(img_stack, dtype=torch.float32)
```

[수정 후 코드]

```
img_tensor = torch.tensor(img_stack, dtype=torch.float32)
```

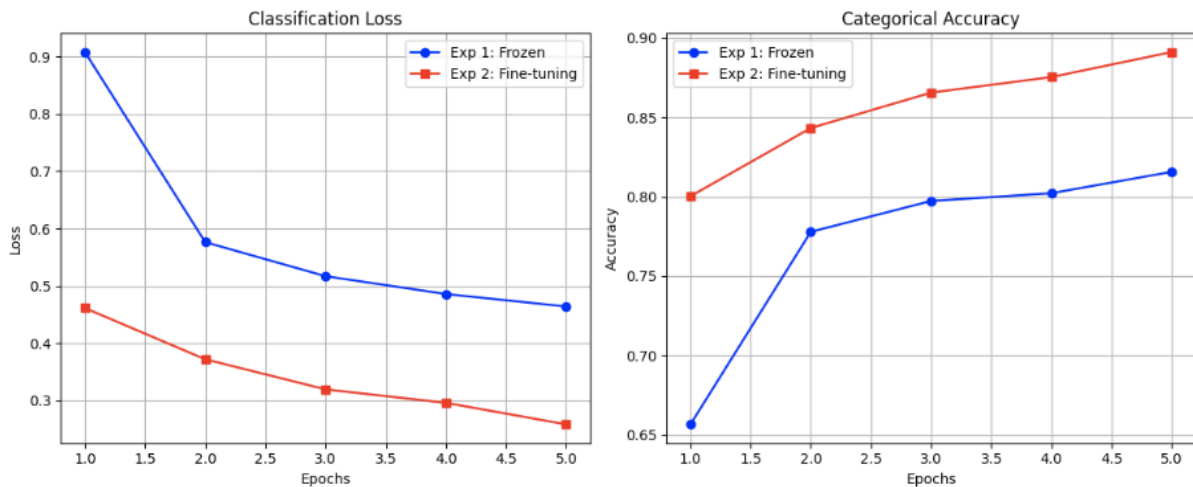
```
img_tensor = (img_tensor - img_tensor.min()) / (img_tensor.max() - img_tensor.min())
```

3.3 수정 결과

- 데이터의 분포가 0.0 ~ 1.0으로 균일하게 맞춰짐으로써 모델이 CWT 이미지의 패턴(Texture)을 명확하게 학습할 수 있게 되었습니다.
- 그 결과, 리더보드 점수가 0.6189 → 0.8025로 약 30% 성능 향상을 달성했습니다.

4. 실험 결과

4.1 학습 곡선 (Loss & Accuracy Graph)



- 파란색 선 (Freeze): Loss가 느리게 감소하고 Accuracy가 낮음.
- 빨간색 선 (Fine-tuning): Loss가 급격히 감소하며 높은 Accuracy에 도달함.

4.2 최종 성능

- 제출 파일: submission.csv (실험 2의 Fine-tuning 모델로 추론)
- Acc 베이스라인 점수: 0.7397
- 나의 최종 점수: 0.8025 (베이스라인 대비 +0.0628 달성)

5. 실험 분석

본 프로젝트에서 수행한 두 가지 실험(동결 vs 미세조정)의 결과를 비교 분석하면 다음과 같습니다.

5.1 실험 1: CNN 부분 동결

설정: CNN의 가중치를 고정(requires_grad=False)하고 마지막 FC Layer만 학습했습니다.

- 결과: 정확도가 상대적으로 낮고 학습이 더디게 진행되었습니다.
- 이유: Pretrained ResNet18은 일반적인 '자연 이미지(강아지, 고양이 등)'의 특징을 추출하도록 학습되어 있습니다. 본 프로젝트의 'CWT 변환된 센서 신호 이미지'는 자연 이미지와 시각적 패턴(Texture, Edge 등)이 매우 다릅니다. 따라서 기존 필터를 그대로 사용하면 센서 데이터의 고유한 특징을 제대로 잡아내지 못합니다.

5.2 실험 2: 미세 조정 (Fine-tuning)

- 설정: 모델의 모든 파라미터(requires_grad=True)를 업데이트하며 학습했습니다.
- 결과: 실험 1에 비해 월등히 높은 정확도와 빠른 Loss 수렴을 보였습니다.
- 이유: 역전파를 통해 CNN의 필터들이 센서 데이터(CWT)의 패턴에 맞게 재조정되었습니다. 이는 모델이 일반적인 이미지 특징이 아닌, 환자의 활동 상태를 나타내는 주파수/시간적 특징을 효과적으로 학습했음을 의미합니다.

6. 결론

본 과제를 통해 IMU 센서 데이터를 이미지 모델(ResNet)에 적용하는 파이프라인을 구축했습니다. 특히 단순히 모델을 가져다 쓰는 것을 넘어, 데이터의 특성(Scale)을 파악하고 적절한 전처리를 적용하는 것이 모델 성능에 결정적인 영향을 미친다는 점을 확인했습니다. 결과적으로 미세 조정(Fine-tuning)과 정규화를 통해 베이스라인을 상회하는 0.8025의 높은 정확도를 달성했습니다.

7. 전체 코드

```
import pandas as pd
import numpy as np
import random
import os
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import models

#[1]Random Seed 고정
SEED = 42
BATCH_SIZE = 32
EPOCHS = 5
LEARNING_RATE = 1e-3
RESIZE_SHAPE = (224, 224)
NUM_CLASSES = 6
SELECTED_SIGNAL = 'acc'

def seed_everything(seed):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True

seed_everything(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}, Seed: {SEED}")

DATA_PATH = '/kaggle/input/2025-ai-project-1'

#[2]데이터셋 정의
class SensorDataset(Dataset):
    def __init__(self, data_path, mode='train', signal_type='acc'):
        self.mode = mode
        self.signal_type = signal_type.lower()
```

```

#파일명 찾기 로직
def find_file(path, keyword_start, keyword_end):
    for filename in os.listdir(path):
        if filename.lower().startswith(keyword_start.lower()) and \
            filename.lower().endswith(keyword_end.lower()):
            return os.path.join(path, filename)
    return None

suffix = f"{mode}_CWT.csv"

print(f"[{mode.upper()}] Loading {self.signal_type} (x,y,z) data...")
try:
    path_x = find_file(data_path, f"{self.signal_type}_x", suffix)
    path_y = find_file(data_path, f"{self.signal_type}_y", suffix)
    path_z = find_file(data_path, f"{self.signal_type}_z", suffix)

    if path_x is None or path_y is None or path_z is None:
        raise FileNotFoundError(f"파일을 찾을 수 없습니다. 경로: {data_path}")

    self.df_x = pd.read_csv(path_x)
    self.df_y = pd.read_csv(path_y)
    self.df_z = pd.read_csv(path_z)

except FileNotFoundError as e:
    print(e)
    raise

self.data_x = self.df_x.values
self.data_y = self.df_y.values
self.data_z = self.df_z.values

#라벨 로드 (Train 모드일 때만)
if self.mode == 'train':
    #yTrain 파일 찾기
    label_file_path = find_file(data_path, "yTrain", ".csv")
    if label_file_path is None:
        label_file_path = find_file(data_path, "ytrain", ".csv")

    if label_file_path:
        label_df = pd.read_csv(label_file_path)

        self.labels = label_df['Unnamed: 1'].values

        #라벨이 1~6이면 0~5로 변환(PyTorch 학습용)
        if self.labels.min() == 1:
            self.labels = self.labels - 1

print(f"Loaded {len(self.data_x)} samples.")

def __len__(self):
    return len(self.data_x)

def __getitem__(self, idx):
    img_x = self.data_x[idx].reshape(50, 128)
    img_y = self.data_y[idx].reshape(50, 128)
    img_z = self.data_z[idx].reshape(50, 128)

    img_stack = np.stack([img_x, img_y, img_z], axis=0)
    img_tensor = torch.tensor(img_stack, dtype=torch.float32)

```

```

        img_tensor = (img_tensor - img_tensor.min()) / (img_tensor.max() -
img_tensor.min())

        img_resized = F.interpolate(
            img_tensor.unsqueeze(0),
            size=RESIZE_SHAPE,
            mode='bilinear',
            align_corners=False
        ).squeeze(0)

        if self.mode == 'train':
            return img_resized, torch.tensor(self.labels[idx], dtype=torch.long)
        else:
            return img_resized

```

#[3]모델 준비 함수

```

def get_model(freeze_backbone=False):
    model = models.resnet18(pretrained=True)

    if freeze_backbone:
        for param in model.parameters():
            param.requires_grad = False

    num_fters = model.fc.in_features
    model.fc = nn.Linear(num_fters, NUM_CLASSES)

    return model.to(device)

```

#[4]학습 함수

```

def train_experiment(freeze_mode, train_loader, epochs):
    model = get_model(freeze_backbone=freeze_mode)
    optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
lr=LEARNING_RATE)
    criterion = nn.CrossEntropyLoss()

    loss_history = []
    acc_history = []

    model.train()
    for epoch in range(epochs):
        running_loss = 0.0
        correct = 0
        total = 0

        loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}")
        for inputs, labels in loop:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)

```

```

        correct += (predicted == labels).sum().item()

    loop.set_postfix(loss=loss.item())

    epoch_loss = running_loss / len(train_loader)
    epoch_acc = correct / total

    loss_history.append(epoch_loss)
    acc_history.append(epoch_acc)
    print(f"    [Result] Loss: {epoch_loss:.4f}, Acc: {epoch_acc:.4f}")

    return model, loss_history, acc_history

#[5] 메인 실행 (실험 1 & 2)
print(f"Signal Type Selected: {SELECTED_SIGNAL}")
train_dataset = SensorDataset(DATA_PATH, mode='train', signal_type=SELECTED_SIGNAL)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

print("\n===== [실험 1] CNN 부분 동결 =====")
_, loss1, acc1 = train_experiment(freeze_mode=True, train_loader=train_loader,
                                  epochs=EPOCHS)

print("\n===== [실험 2] 미세 조정 (Fine-tuning) =====")
final_model, loss2, acc2 = train_experiment(freeze_mode=False,
                                             train_loader=train_loader, epochs=EPOCHS)

#[6]결과 그래프 시각화
epochs_range = range(1, EPOCHS + 1)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, loss1, 'b-o', label='Exp 1: Frozen')
plt.plot(epochs_range, loss2, 'r-s', label='Exp 2: Fine-tuning')
plt.title('Classification Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(epochs_range, acc1, 'b-o', label='Exp 1: Frozen')
plt.plot(epochs_range, acc2, 'r-s', label='Exp 2: Fine-tuning')
plt.title('Categorical Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

#[7]추론 및 제출 파일 생성
test_dataset = SensorDataset(DATA_PATH, mode='test', signal_type=SELECTED_SIGNAL)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

final_model.eval()
predictions = []

```

```
with torch.no_grad():
    for inputs in tqdm(test_loader, desc="Inference"):
        inputs = inputs.to(device)
        outputs = final_model(inputs)
        _, predicted = torch.max(outputs, 1)
        predictions.extend((predicted.cpu().numpy() + 1).tolist())

submission = pd.read_csv(os.path.join(DATA_PATH, 'submission_sample.csv'))
submission['label'] = predictions
submission.to_csv('submission.csv', index=False)
print("'submission.csv' saved.")
```